

High-Level Design (HLD)

IntelliPlant AI

AI-Powered Industrial Knowledge Intelligence Platform

Version: 1.0

Document Type: High-Level Design (HLD)

Prepared For: ET AI Hackathon 2026

Prepared By: Team IntelliPlant AI

1. Purpose

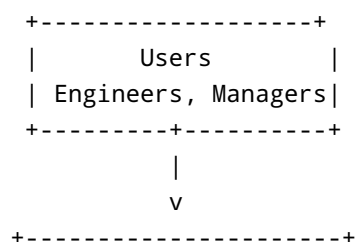
The High-Level Design (HLD) document describes the overall structure of IntelliPlant AI and explains how major modules interact with each other. It serves as the bridge between the System Architecture Document (SAD) and the Low-Level Design (LLD).

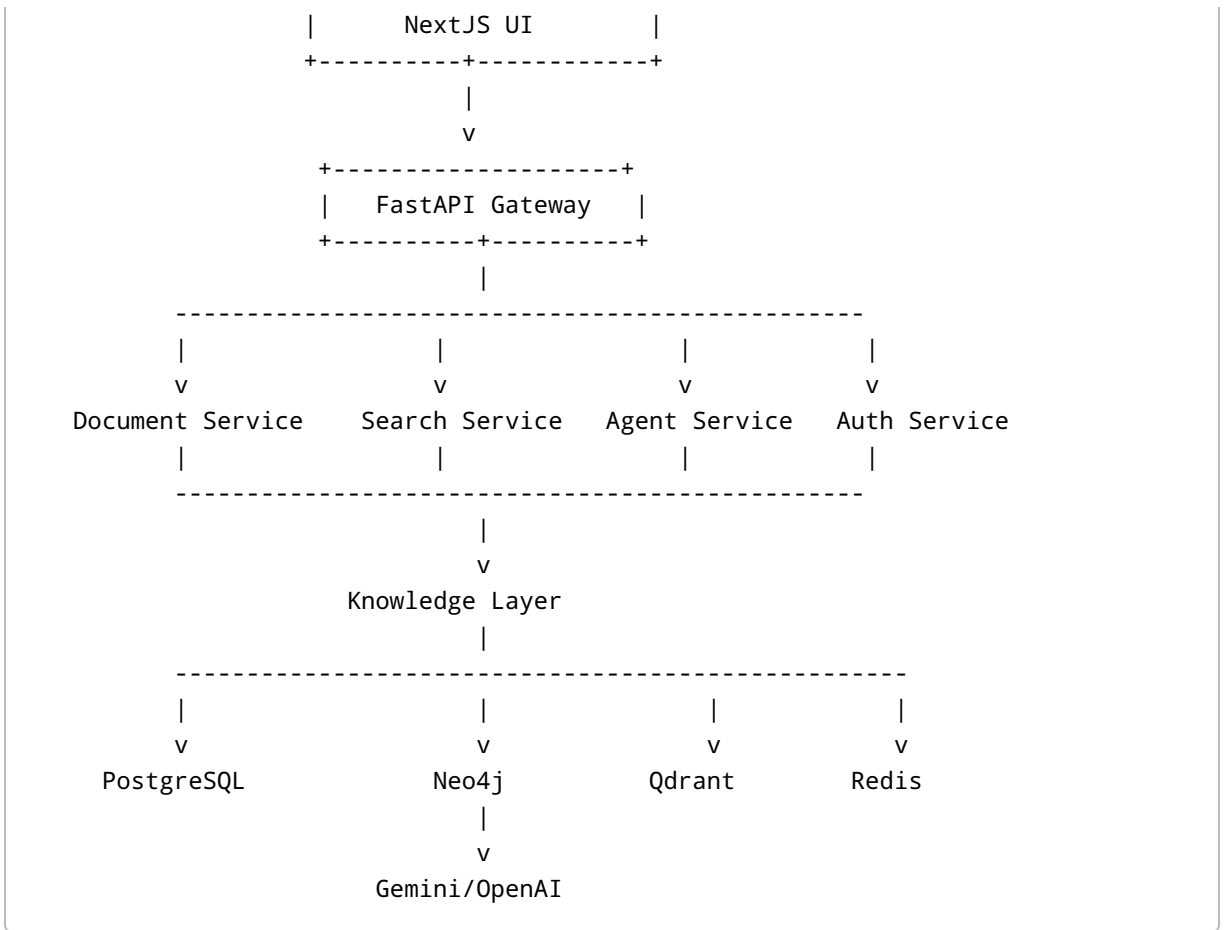
2. Design Goals

The system should provide:

- Scalability
 - Explainability
 - Modularity
 - Fault tolerance
 - Maintainability
 - High performance
 - Extensibility
-

3. System Context Diagram





4. Layered Architecture



5. Major Components

Frontend Layer

Technology

- NextJS
- TypeScript
- TailwindCSS

Responsibilities

- Dashboard
 - Chat Interface
 - Graph Visualization
 - Document Explorer
-

API Layer

Technology

FastAPI

Responsibilities

- Authentication
 - Request Routing
 - Validation
 - Response Formatting
-

Business Layer

Responsible for:

- Search
 - Document Processing
 - User Management
 - Audit Logs
-

AI Layer

Responsible for:

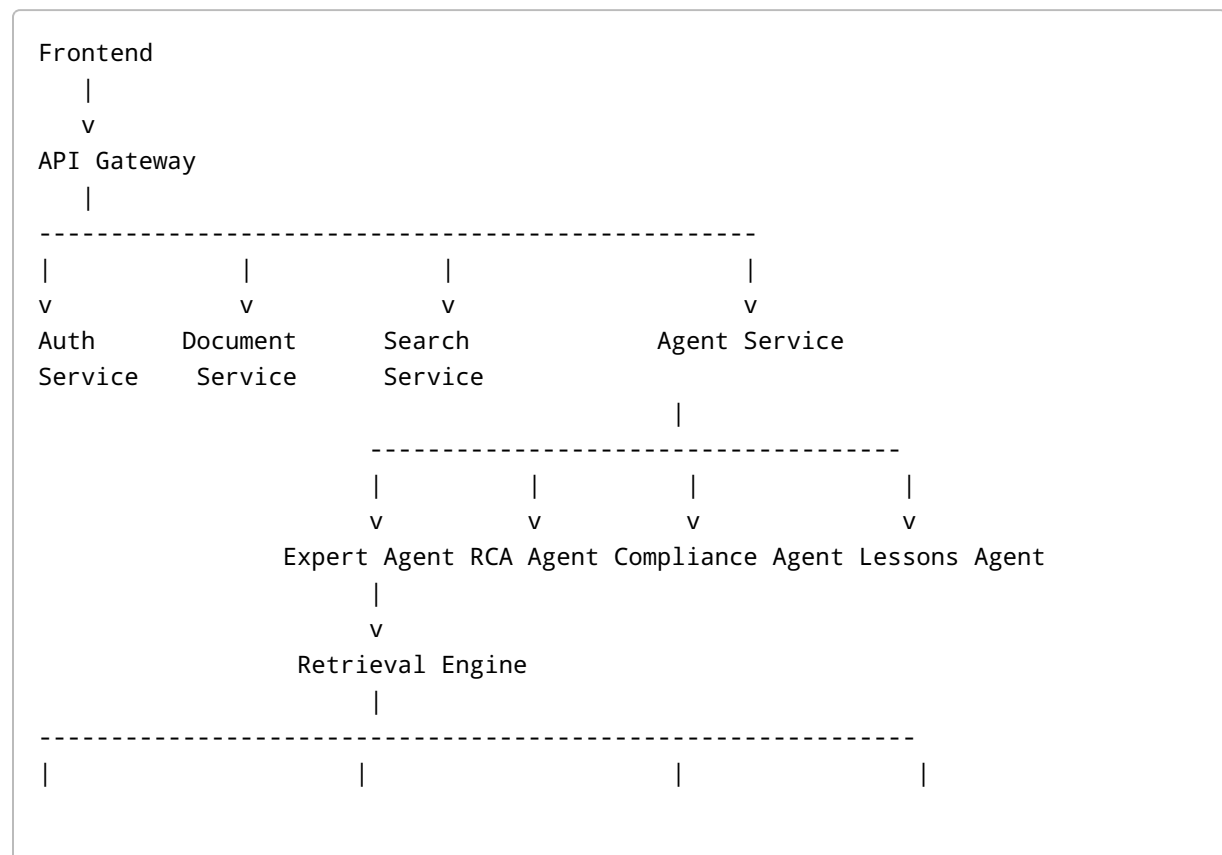
- RAG
- Agent Orchestration
- Query Planning
- Reasoning

Knowledge Layer

Responsible for:

- Metadata
- Vector Search
- Graph Search

6. Component Diagram



v
Qdrant

v
Neo4j

v
PostgreSQL

v
Redis

7. Functional Modules

Authentication Module

Responsible for:

- Login
- JWT
- RBAC

Document Module

Responsible for:

- Upload
- OCR
- Parsing
- Metadata extraction

Knowledge Graph Module

Responsible for:

- Node generation
- Relationship extraction

Embedding Module

Responsible for:

- Chunking
 - Vector generation
-

Search Module

Responsible for:

- Hybrid retrieval

Multi-Agent Module

Responsible for:

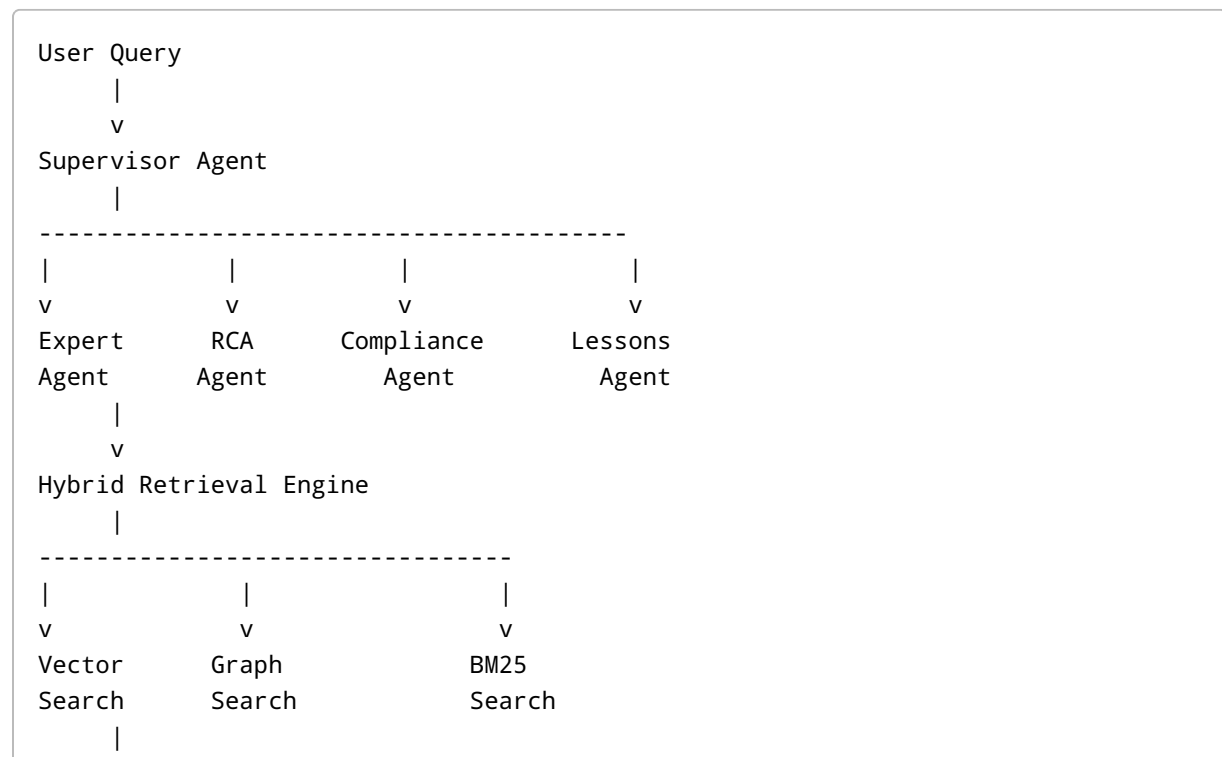
- Task routing
- Query execution

Dashboard Module

Responsible for:

- Analytics
- Visualization

8. AI Component Architecture



v
Gemini/OpenAI

9. Data Flow Diagram

Document Processing Flow

```
graph TD;
    Upload --> OCR;
    OCR --> MetadataExtraction[Metadata Extraction];
    MetadataExtraction --> EntityExtraction[Entity Extraction];
    EntityExtraction --> Chunking;
    Chunking --> EmbeddingGeneration[Embedding Generation];
    EmbeddingGeneration --> Qdrant;
    Qdrant --> KnowledgeGraphBuilder[Knowledge Graph Builder];
    KnowledgeGraphBuilder --> Neo4j;
```

Question Answering Flow

```
graph TD;
    UserQuery[User Query] --> SupervisorAgent[Supervisor Agent];
    SupervisorAgent --> IntentDetection[Intent Detection];
    IntentDetection --> HybridSearch[Hybrid Search];
    HybridSearch --> AgentSelection[Agent Selection];
    AgentSelection --> LLM[LLM];
    LLM --> CitationEngine[Citation Engine];
```

↓
Response

10. C4 Model

Level 1

System Context

Users interact with IntelliPlant AI.

Level 2

Containers

- Web UI
 - Backend APIs
 - AI Services
 - Databases
-

Level 3

Components

- Search Engine
 - Agent Engine
 - Knowledge Graph
 - OCR Pipeline
-

Level 4

Code-level components

Defined in LLD.

11. Service Interaction Diagram

```
User
↓
Frontend
↓
FastAPI
↓
LangGraph Supervisor
↓
Search Engine
↓
Qdrant + Neo4j
↓
Gemini/OpenAI
↓
Response
```

12. Sequence Diagram

Query Execution

```
User
|
Frontend
|
API Gateway
|
Supervisor Agent
|
Hybrid Search
|
Qdrant
|
Neo4j
|
LLM
|
Response
```

13. Database Architecture

PostgreSQL

Stores

- Users
 - Documents
 - Metadata
 - Audit Logs
-

Neo4j

Stores

- Nodes
 - Relationships
-

Qdrant

Stores

- Embeddings
-

Redis

Stores

- Cache
 - Session State
-

MinIO

Stores

- Original Files
-

14. Deployment Diagram

Docker Environment

Frontend Container

Backend Container

Worker Container

Redis Container

Neo4j Container

PostgreSQL Container

Qdrant Container

MinIO Container

Gemini/OpenAI APIs

15. Security Components

Authentication

JWT

Authorization

RBAC

Encryption

TLS

Audit Logging

Enabled

Input Validation

Pydantic

Rate Limiting

Enabled

16. Background Processing

Technology

Celery + Redis

Tasks

- OCR
- Chunking
- Embeddings
- Graph Construction
- Notifications

17. Error Handling

OCR Failure

Retry queue

Embedding Failure

Fallback worker

LLM Failure

Cached response

Database Failure

Recovery mechanism

18. Scalability

Horizontal scaling

Containerized deployment

Load balancing

Distributed search

Cache layer

19. Performance Targets

Metric	Target
Search latency	<2 sec
Chat response	<5 sec
OCR processing	<20 sec
Upload time	<30 sec
Concurrent users	1000+
Uptime	99%

20. Technology Stack

Frontend

- NextJS
- TypeScript
- TailwindCSS

Backend

- FastAPI
- Celery
- LangGraph

Databases

- PostgreSQL
- Neo4j
- Qdrant
- Redis

AI

- Gemini
- OpenAI
- BGE Embeddings

OCR

- Docling
- PaddleOCR

Deployment

- Docker
 - Kubernetes
 - AWS
 - Azure
-

21. Future Extensions

- SAP Integration
 - SCADA Integration
 - IoT Sensors
 - Digital Twin
 - Predictive Maintenance
 - Voice Copilot
 - Mobile Application
-

Conclusion

IntelliPlant AI follows a modular, layered, and scalable architecture centered around GraphRAG and Multi-Agent AI. The HLD provides a high-level blueprint for implementing the platform while maintaining flexibility, reliability, and future extensibility.